

CPU Governor Configuration, Frequency, and Offline RL Control

A Distilled Supervised Expert Pipeline and an Offline Reinforcement Learning Comparison

Contents

1. Executive Summary
2. System Overview and Methodology
3. The Supervised Expert System
 - 3.1 Why Two Separate Models?
 - 3.2 The Configuration Model
 - 3.3 The Frequency Model
 - 3.4 The Final Expert Pipeline: Chaining the Two Students
4. Offline Reinforcement Learning
 - 4.1 Motivation: Why Explore RL As Well?
 - 4.2 Problem Formulation: An Offline Contextual Bandit
 - 4.3 Why DQN / Double DQN
 - 4.4 Reward Design
 - 4.5 Training Workflow
 - 4.6 Results: Regret vs. Oracle, Expert, and Ondemand
 - 4.7 Results: Frequency-Imitation Quality
 - 4.8 Key Finding: Action-Space Collapse in the Continuous Agents
 - 4.9 Results: Latency
5. Consolidated Latency Summary
6. Validation and Integrity Checks
7. Conclusions and Recommendations
- Appendix A: Scope Notes and Omissions

1. Executive Summary

This report documents a governor-configuration and CPU-frequency control system built from measured hardware data from the PPDS M/M/1 dataset, and compares two ways of arriving at a control policy: a supervised expert-and-student pipeline, and six offline reinforcement learning agents trained on the same data. All results below come from a single, unified, leakage-safe leave-one-state-out evaluation protocol, so every number in this report is directly comparable across models.

The supervised pipeline is built from two purpose-specific models chained together. A configuration model chooses the governor's sampling rate and utilization threshold to minimize measured Energy-Delay Product (EDP); a frequency model then predicts the operating frequency that configuration will produce. Each model exists as a slower, transparent “expert” and a distilled, latency-optimized “student” that reproduces the expert's decisions almost exactly. The deployed system (the Final Expert Pipeline) chains the two students: it selects a configuration in roughly 8 microseconds and, from that same decision, produces a frequency estimate in a further ~50 microseconds, for a combined 0.14 milliseconds end to end. It closes 19.09 of the 24.02 percentage points of average EDP improvement available between the Linux ondemand governor and a theoretical oracle — about 79% of the maximum possible gain — and it beats ondemand's own EDP in 10 of 10 held-out utilization states.

Six offline reinforcement learning agents were trained on the same measurements and evaluated under the same protocol, split into two families: three discrete Double-DQN variants (including a purpose-built variant that also learns a binned frequency decision) and three continuous actor-critic methods (DDPG, TD3, SAC). None of the six agents beat the supervised expert outright, but all six clearly beat the Linux ondemand default. The discrete DQN family came within about 7–8 percentage points of the expert's regret and recovered roughly 85–92% of the expert's advantage over ondemand; the three continuous agents recovered far less — only about 6–21% of that advantage — because, as this report documents in detail, they converged to a near-constant, high-sampling-rate configuration policy that stopped meaningfully responding to the utilization state.

Every model in the system, from the slowest expert to the fastest RL agent, decides in well under half a millisecond, so latency is not a constraint on deploying any of them. The remaining sections justify every architectural choice in detail, report full accuracy and latency figures for every model, and lay out the offline RL methodology and results.

2. System Overview and Methodology

2.1 Problem Definition

The Linux CPU frequency governor periodically samples processor utilization and adjusts operating frequency according to two tunable parameters: the sampling rate (how often it polls) and the utilization threshold (the trigger point for scaling up). The default “ondemand” governor uses a fixed configuration (a 10,000-microsecond sampling rate and a 95% threshold) regardless of workload. This project asks whether a learned controller can choose a better configuration for the workload actually being observed, and can further anticipate the operating frequency that configuration will produce, while remaining cheap enough to run inline on every scheduling decision.

The optimization target throughout is the Energy-Delay Product (EDP), defined as measured mean response time squared multiplied by measured core power ($EDP = MRT^2 \times Power$). EDP is the standard way of jointly penalizing both wasted energy and degraded responsiveness in a single scalar, and it is only meaningfully defined where power was actually measured on hardware, which in this dataset is the x86 platform. Configuration selection, the offline RL agents, and the Final Expert Pipeline's decision are therefore all scoped to x86. The frequency model itself is trained and evaluated on both x86 and ARM, since frequency telemetry (independent of power measurement) exists for both platforms.

2.2 Data

The dataset comprises 9,979 measured runs across the two platforms (5,000 ARM, 4,979 x86), spanning a common action grid of 50 governor configurations — 10 sampling rates crossed with 5 thresholds. On x86, after joining measurements to a target-utilization grid, this yields a configuration table of 500 rows (10 utilization states $\times$ 50 actions) with EDP computed for every cell, so every candidate configuration's outcome is known at every state — not just the configuration that was actually deployed. This “full information” property is what makes rigorous leave-one-state-out evaluation possible, and it is also what makes the offline RL stage a contextual bandit rather than a sparse-reward problem, discussed further in Section 4.2. The frequency table spans both architectures, 1,000 rows (20 states $\times$ 50 actions).

2.3 Evaluation Protocol

Every model in this report — supervised and RL alike — is evaluated with leave-one-state-out cross-validation (LOOCV): for each held-out utilization state, every artifact needed to produce a decision for that state (curve fits, distilled tables, trained networks, reward surfaces, even hyperparameter choices) is built using only the other states, and the held-out state's own ground-truth measurement is read only after a decision has already been committed. No exceptions were made anywhere in the pipeline, including inside the RL agents' own inner-fold hyperparameter search (Section 4.5). This is what makes every accuracy number in this report an honest estimate of performance on a workload state the model has not seen, and what makes the supervised and RL numbers directly comparable to one another.

Regret, the primary accuracy metric used throughout, is defined per decision as $(\text{EDP achieved} - \text{EDP of the oracle-best configuration}) / \text{EDP of the oracle-best configuration}$, so 0% regret means the model matched the best possible measured outcome for that state and larger values mean proportionally worse energy-delay behavior. The same formula, substituting the ondemand governor's EDP for the achieved EDP, defines ondemand's own regret and is reported throughout as the baseline every other policy is measured against.

3. The Supervised Expert System

3.1 Why Two Separate Models?

Configuration selection and frequency estimation are deliberately built and validated as two separate models rather than one joint model, because they answer two different kinds of question against two different kinds of evidence. The configuration decision — which sampling rate and threshold to apply — is optimized against a directly measured, ground-truth objective: EDP was physically measured on hardware for every one of the 50 candidate configurations at every utilization state, so there is a real reward signal to fit against and to score decisions with. Frequency, by contrast, was never independently commanded in these measurements; it is the processor's own response after a configuration has already been applied, observed rather than chosen. There is no measured EDP outcome for “a different frequency at the same configuration”, so frequency cannot honestly be optimized the way configuration can — it can only be predicted.

Collapsing these into a single model would force one of two compromises: fabricating an EDP-based objective for frequency that does not exist in the data, or blending a discrete, reward-optimized decision with a continuous, observation-fitted regression inside one architecture, which is harder to train, harder to validate independently against the right ground truth, harder to interpret, and harder to distill for latency. Keeping them separate lets each stage use the model family suited to its own problem, be checked against its own ground truth, and be re-fit independently if either the configuration measurements or the frequency measurements are extended in the future, without disturbing the other.

3.2 The Configuration Model

3.2.1 Model choice: PCHIP curves per candidate configuration

The configuration expert fits one Piecewise Cubic Hermite Interpolating Polynomial (PCHIP) curve per candidate configuration, mapping utilization to predicted EDP (on a $\log_{10}(1 - p)$ input transform, relative to the ondemand curve), and at decision time evaluates all 50 curves and takes the arg-min. PCHIP was chosen over a smoothing spline or a black-box regressor for three concrete reasons. First, it is shape-preserving: unlike a plain cubic spline, it will not overshoot or oscillate between measured points, which matters because EDP spans more than three orders of magnitude across the utilization range (from roughly 19,000 at 10% utilization to over 11,000,000 near saturation) — an overshooting interpolant could invent a spurious “best” configuration in a region with no supporting data. Second, fitting one interpretable curve per configuration, rather than a single joint classifier, keeps the decision rule fully transparent and auditable: there is no opaque model to inspect, only 50 curves and an arg-min, and every curve can be plotted and sanity-checked directly (Figure 1). Third, it needs no hyperparameter tuning beyond the input transform, minimizing overfitting risk against a modest x86 dataset of 10 utilization states per fold.

3.2.2 Why distillation: the lookup-table student

A governor decision has to be made inline, on the target device, on every scheduling tick — not in an offline analysis environment. Evaluating 50 independent PCHIP curves and taking an arg-min measured 0.28 milliseconds of median latency per decision (Section 3.2.3), which is both wasted CPU work and a latency tail that is hard to bound in a real-time control loop. The expert's decision surface was therefore pre-computed once, offline, across a dense 4,096-bin lookup table (LUT) spanning the observed utilization range — a bin count carried over unchanged from the bin-size search performed in the original modeling work. At runtime, inference collapses to a single nearest-bin lookup: no interpolation and no arg-min. This distillation step is a compression of the expert's already-validated decision surface, not a retrained or approximated model, which is why the student's accuracy is identical to its teacher's to four decimal places, and why it agrees with a fresh, fold-local re-fit of the teacher on 99.93% of a dense sweep of the utilization range (99.90% in the worst fold) — the 0.07% residual disagreement occurs only exactly at bin boundaries where two configurations are within noise of each other.

3.2.3 Accuracy and latency results

Table 1 reports leave-one-state-out regret against the oracle for every configuration policy, across all 10 held-out x86 utilization states, together with the average EDP improvement over the Linux ondemand governor and the share of states in which each policy beat ondemand outright.

Policy	Mean regret	Median regret	Worst regret	Exact-oracle rate	Within 5% of oracle	Avg. EDP vs. ondemand	Beats ondemand
Oracle (theoretical best)	0.00%	0.00%	0.00%	100%	100%	−24.02%	100%
Configuration Expert (PCHIP)	6.93%	3.90%	19.52%	30%	60%	−19.09%	100%
Configuration Student (4096-bin LUT)	6.93%	3.90%	19.52%	30%	60%	−19.09%	100%
Linux Ondemand (fixed default)	56.64%	23.37%	365.24%	0%	0%	0.00%	—

The expert and student are identical to four decimal places on every metric — distillation lost no accuracy. Both close 19.09 of the 24.02 percentage points of average EDP improvement that separate ondemand from the oracle (about 79% of the maximum achievable gain), and both beat ondemand's measured EDP in every one of the 10 held-out states. Ondemand's own worst-case regret, 365.24%, occurs at full load ($p = 0.98$), where its fixed configuration is furthest from the state-appropriate choice — this is the single largest driver of ondemand's poor average.

Model	Median latency	p95 latency	p99 latency	Size	Speedup vs. expert
Configuration Expert (PCHIP teacher)	0.2844 ms	0.3856 ms	0.4761 ms	24.93 KB	1.0×
Configuration Student (4096-bin LUT)	0.0083 ms	0.0133 ms	0.0284 ms	36.39 KB	34.3×

Table 2. Configuration model latency, single decision, measured over 300 repeated calls with warmup.

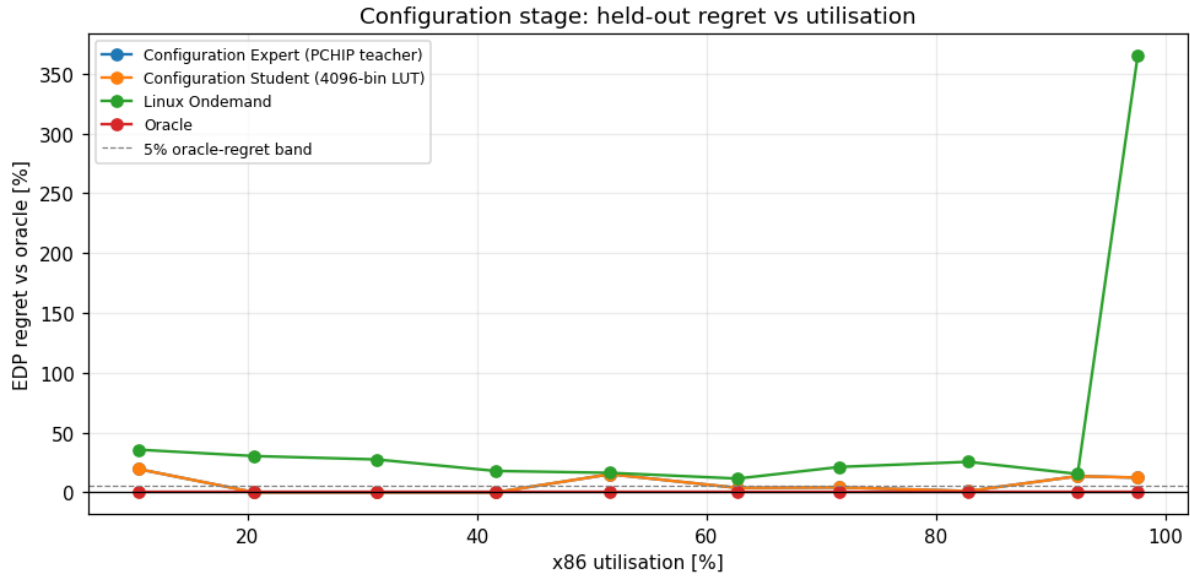


Figure 1. EDP regret vs. utilization for every configuration policy. The expert and student curves overlap exactly; ondemand's regret grows sharply as utilization approaches saturation.

3.3 The Frequency Model

3.3.1 Model choice: RBF-SVR

The frequency expert is a Support Vector Regressor with a radial basis function kernel ($C=100$, $\gamma=0.1$, $\epsilon=0.005$ — carried over unchanged from the original frequency-modeling notebook and not re-tuned for this report), wrapped with feature standardization. RBF-SVR was chosen because the mapping from utilization and configuration to operating frequency is smoothly nonlinear without a known closed form the way EDP's curve shape is, so a flexible nonparametric kernel method is a better fit than committing to a particular functional form. Its epsilon-insensitive loss also gives it a degree of built-in robustness to ordinary measurement noise in frequency telemetry, and it performs well on the moderate sample sizes available per architecture (500 x86 rows, 500 ARM rows) without the larger data requirements of a bigger neural network. Predictions are produced in a normalized $[0,1]$ frequency space and denormalized through each architecture's own legal frequency bounds (800 MHz–2.1 GHz for x86, 600 MHz–2.4 GHz for ARM), so every output is legal by construction. A convergence audit across all 20 leave-one-state-out folds (both architectures) confirmed the solver reached full convergence in every fold with zero iteration-cap hits, so the teacher being distilled from in the next step is a reliably fitted one.

3.3.2 Why the frequency model depends on the configuration model's output

Operating frequency is jointly determined by utilization and by which governor configuration is currently active: at the same target utilization, a governor polling infrequently with a high threshold drives the processor through a different transient response than one polling aggressively with a low threshold, and the two settle at measurably different frequencies. This is why the chosen configuration's sampling rate and threshold — the output of the first model — are passed forward as explicit input features to the frequency model, alongside utilization and architecture. The full input feature set, and the reason for each one, is: p

(target utilization, the primary driver of frequency demand); $\log_{10}(1-p)$ (the same saturation-focused transform used successfully in the configuration model, giving the model more resolution exactly where the response curve is steepest); the chosen configuration's normalized sampling rate and threshold (the causal link to the first model's decision, justified below); a one-hot architecture flag (ARM and x86 occupy different absolute frequency ranges and, as shown next, respond differently to configuration); and the architecture's normalized legal frequency bounds (so the model is aware of its own valid output range and predictions can be denormalized safely).

This causal dependency was verified directly, not assumed, with a config-blind ablation: a version of the model with the configuration features removed, scored against the same held-out ground truth. On ARM, removing configuration awareness raises mean absolute error from 16.87 MHz to 72.33 MHz — a skill ratio of 0.23, meaning the configuration-aware model's error is less than a quarter of the config-blind version's, a clear and substantial benefit. On x86, the same ablation gives a skill ratio of 1.93: the configuration-aware model is marginally worse, not better, than the config-blind version. This is an expected and honest architecture-specific result rather than a flaw — x86's measured frequency response is nearly flat across the whole action grid in this dataset (roughly a 40 MHz / 8 MHz spread, close to the measurement noise floor), so there is very little configuration-driven signal for the model to exploit there, and including the feature costs a small amount of estimation variance instead of buying accuracy. The configuration features were kept for both architectures for architectural consistency and because they deliver a clear, verified benefit on ARM while costing negligibly little on x86.

3.3.3 Why distillation: the MLP student

The same inline-latency argument that motivated the configuration student applies here: the RBF-SVR expert must evaluate a kernel function against every support vector at prediction time, which measured 0.25 milliseconds of median latency. A small feed-forward network (two hidden layers of 32 units each, tanh activations, trained with Adam) was distilled from the expert by training on a dense re-sampling of the expert's own predicted surface (48 utilization points per candidate configuration) rather than on the raw measurements directly. This reduces median latency to 0.050 milliseconds — a 5.1× speedup — while, notably, slightly improving held-out accuracy over the teacher (16.85 MHz mean absolute error versus the teacher's 29.91 MHz). This is not a paradox: both models are scored against the same held-out ground-truth measurements, not against each other, and training on a smooth, densely re-sampled version of the teacher's surface acts as a mild regularizer that generalizes slightly better under this evaluation protocol than the teacher's own direct fit to noisier raw measurements. Measured directly against the teacher's own predictions (rather than against ground truth), the student differs by 18.58 MHz mean absolute error, confirming it is a faithful distillation of the teacher's surface and not an independently lucky fit.

3.3.4 Accuracy and latency results

Model	MAE	RMSE	R ²	MAPE	Legal violations
Frequency Expert (RBF-SVR)	29.91 MHz	59.47 MHz	0.925	1.53%	0%
Frequency Student (MLP 32×32)	16.85 MHz	29.23 MHz	0.982	0.91%	0%

Table 3. Frequency model accuracy, leave-one-state-out, both architectures combined (20 folds). The student differs from the teacher by 18.58 MHz MAE when compared directly.

Platform	Student MAE (config-aware)	Config-blind MAE	Skill ratio (< 1 ⇒ configuration helps)
ARM	16.87 MHz	72.33 MHz	0.23
x86	16.83 MHz	8.71 MHz	1.93

Table 4. Config-blind ablation, confirming the frequency model's dependence on the configuration decision is real and architecture-specific.

Model	Median latency	p95 latency	p99 latency	Size	Speedup vs. expert
Frequency Expert (RBF-SVR)	0.2549 ms	0.3667 ms	0.4787 ms	28.50 KB	1.0×
Frequency Student (MLP 32×32)	0.0503 ms	0.0798 ms	0.0933 ms	13.02 KB	5.1×

Table 5. Frequency model latency, single prediction.

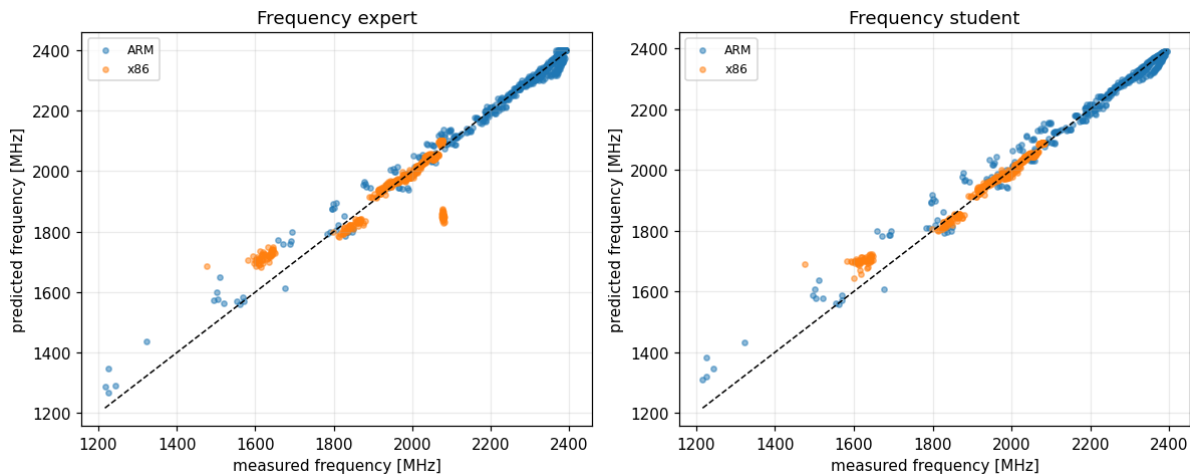


Figure 2. Predicted vs. measured frequency, expert and student, both architectures. Points near the diagonal indicate accurate prediction.

3.4 The Final Expert Pipeline: Chaining the Two Students

The deployed system composes only the two distilled students — never the slower teachers — into a single inline decision function. Given a utilization level, the configuration student selects a sampling rate and threshold; that decision, together with utilization and the architecture flag, is immediately passed into the frequency student, which returns the predicted operating frequency. This mirrors the causal dependency established in Section 3.3.2 and returns one complete, physically consistent decision per call: what

configuration to apply, and what frequency to expect from applying it. Chaining the two students, rather than pairing a fast student with a slow teacher, keeps the combined decision inline-fast.

Because the configuration student's decision is computed first and nothing in the frequency stage feeds back into it, attaching the frequency prediction can never change which configuration gets selected — this additivity property was verified directly rather than assumed: the pipeline's measured EDP regret (6.93% mean) is identical to the standalone configuration student's regret to four decimal places, and this equality is enforced as an automated check on every run (Section 6), so any future change to the frequency stage can never silently alter configuration accuracy. On the configuration it actually selects, the pipeline's frequency prediction carries a mean absolute error of 18.64 MHz against measured ground truth. As a concrete example, at $p = 0.63$ the pipeline selects configuration 35 (80,000-microsecond sampling rate, 75% threshold) and predicts an operating frequency of approximately 2.017 GHz for that decision.

Metric	Value
Mean EDP regret vs. oracle	6.93% (identical to the configuration student)
Frequency MAE on the selected configuration	18.64 MHz
States beating ondemand's EDP	10 / 10
Median end-to-end latency	0.1401 ms (p95 0.2056 ms, p99 0.2635 ms)
Combined model size	49.24 KB

Table 6. Final Expert Pipeline summary — configuration student and frequency student chained.

4. Offline Reinforcement Learning

4.1 Motivation: Why Explore RL As Well?

The supervised expert-and-student pipeline in Section 3 already performs well, but it depends on a human-designed decision rule (curve-fitting followed by an arg-min) built on top of a hand-chosen objective. Offline reinforcement learning was explored as a complementary approach for three reasons. First, it tests whether a value-based or policy-based learner can recover a comparable policy directly from reward signal, without a hand-authored curve-fitting rule — relevant if the action grid, the reward definition, or the available state features change in the future in ways that would require re-deriving a new curve-fitting approach by hand. Second, it is the natural extension path if the system should eventually keep improving from newly observed data online, rather than being re-fit from scratch each time. Third, and most immediately useful to the research team, it produces an apples-to-apples benchmark for exactly how much of the supervised expert's advantage over the Linux default is recoverable by a more general-purpose learning method — valuable evidence either way, since it either justifies RL as a more flexible replacement or quantifies precisely how much value the hand-designed approach is currently contributing.

4.2 Problem Formulation: An Offline Contextual Bandit

As noted in Section 2.2, the measured dataset is a full-information contextual bandit: for every utilization state, the EDP outcome of all 50 candidate configurations is known, not only the configuration that happened to be deployed, and no decision at one state affects the starting condition of any other state. There is no temporal-credit-assignment problem in this data. Every agent in this section is therefore trained with a discount factor of $\gamma = 0$, which is the honest formulation: it makes every training target reduce to the immediate, measured (or imitated) reward with no bootstrapped future-value term. The Double-DQN bootstrap term is retained in the implementation, multiplied by γ , purely so the same code would generalize correctly if the team later works with genuinely sequential data; for this dataset it always evaluates to zero and contributes nothing to training.

4.3 Why DQN / Double DQN

The configuration decision is inherently categorical, not continuous: ground-truth EDP measurements exist only for 50 specific, discretely enumerated (sampling rate, threshold) pairs, so the only actions a policy can be honestly scored against are picks from that fixed 50-item menu, not a free two-dimensional continuous choice. Value-based methods over a discrete action set — the DQN family — are the standard, natural fit for exactly this kind of problem: a Q-network outputs one score per menu item and is trained directly against a fully-known reward table, an unusually clean setting for value-based RL that is normally complicated by missing data for untried actions.

Double DQN specifically was used rather than vanilla DQN because vanilla Q-learning is well documented to systematically overestimate action values, since the same network both selects and evaluates the greedy

next action, which compounds any noise in the value estimates into an optimistic bias. Double DQN corrects this by decoupling the two roles — an online network selects the greedy action, and a separate, slowly-updated target network evaluates it. This correction is not a nice-to-have in this setting: the offline dataset is small, as few as nine x86 training states per leave-one-out fold, and small-sample value estimates are exactly the noisy regime in which DQN's overestimation bias is worst, so Double DQN's decoupling directly improves the trustworthiness of the learned values here.

Three architectural variants of the Double-DQN approach were trained so the results could be compared directly. The plain variant is a single shared network (48-then-48 tanh hidden units) with one linear output head scoring all 50 configurations, used as the baseline. The Dueling variant shares a trunk but splits into a scalar state-value head and a per-action advantage head, recombined as $Q = V + A - \text{mean}(A)$; this separates “how good is this state overall” from “how much better is this specific action than the state's average”, which tends to help when many actions have similar values — plausible here, since at low utilization many long-sampling-rate configurations are all close to equally good, so isolating the shared state-value component can reduce noise in the per-action estimates. The third variant, the Discretised-frequency (Factorised) DQN, was purpose-built to satisfy the requirement that frequency also be decided as a discrete, binned choice rather than left out of the RL formulation entirely: a shared trunk feeds both the same 50-way configuration head and a second, configuration-conditioned frequency head that scores a fixed number of frequency bins (nested-selected per fold, Section 4.5) — the chosen configuration's normalized sampling rate and threshold are fed into the frequency head as extra context, so the binned frequency decision is explicitly aware of which configuration it is being made for. All three variants share the identical Double-DQN target computation and Polyak-averaged target-network update described in Section 4.5.

Three continuous policy-gradient methods — DDPG, TD3, and SAC — were trained alongside the DQN family as an independent second comparison arm, not as a replacement for the discrete framing but to test whether relaxing the action space to a free continuous vector could match or exceed the discrete agents, since the discretization the DQN family relies on is, in principle, an approximation of an underlying continuous control space. As Section 4.8 documents, this relaxation did not pay off: the continuous agents performed markedly worse, for a specific and identifiable reason.

Agent	Action variables	How the action space is structured
DDQN	1 — configuration only	Single 50-way categorical pick over the pre-enumerated (sampling rate, threshold) grid. No frequency output.
Dueling DDQN	1 — configuration only	Same 50-way action space as DDQN, estimated via a value/advantage (dueling) architecture instead of a plain Q-head.
Discretised-frequency DDQN	2 — configuration, then frequency bin	A configuration head (50 options) and a frequency head (64 bins at deployment) conditioned on the chosen configuration; two sequential discrete decisions.
DDPG / TD3 / SAC	3 — sampling rate, threshold, frequency	The actor outputs [sr_norm, threshold_norm, frequency_norm] directly as one continuous vector in [0,1]; sampling rate and threshold are snapped to the nearest measured grid point only for reward lookup, while frequency remains fully continuous.

Table 7. Action space of each of the six RL agents.

4.4 Reward Design

Configuration decisions, in every agent, are trained against the real, measured EDP-based reward table — the same ground truth the supervised expert is scored against. Frequency decisions — in the Discretised-frequency DDQN's frequency head and in the three continuous agents' third output dimension — are never trained against a fabricated EDP-based reward, because, as established in Section 3.3.2, frequency was never independently commanded in the source measurements and there is no ground truth for what EDP would have resulted from a different frequency at a fixed configuration. Instead, wherever an agent must decide a frequency, it is trained purely as an imitation target: the negative squared distance, in normalized frequency space, to the fold-local supervised Frequency Expert's own prediction, scaled by a weight, `lambda_freq`. This keeps every agent honest — configuration decisions are only ever rewarded for real, measured outcomes, and frequency, when decided at all, is transparently and explicitly a “match the frequency model” objective rather than a discovered one.

4.5 Training Workflow

Every agent follows the identical leave-one-state-out discipline used for the supervised models: for each of the 10 held-out x86 states, the fold-local configuration expert, frequency expert, and reward table used to train that fold's agents are built from the other nine states only, and the held-out state's own measurement is read only after a decision has already been committed. This is what makes the RL numbers in Sections 4.6–4.9 directly comparable to the supervised expert and student numbers in Section 3.

The three continuous actors are given a behavior-cloning warm start before any reward-driven training begins: 500 steps of supervised regression toward the fold-local configuration expert's best decision and the frequency expert's prediction. This is standard practice for offline actor-critic methods, which are otherwise prone to instability when learning entirely from a fixed batch of data with no live environment to explore against; starting from a reasonable policy rather than a randomly initialized one keeps the actor from drifting into action regions the data does not support.

The Discretised-frequency DDQN has two hyperparameters — the number of frequency bins (16, 32, or 64) and the imitation weight `lambda_freq` (0, 0.25, 0.5, 1.0, or 2.0) — that are jointly selected per outer fold via a nested inner search, using only that fold's own training states and never the held-out state. Every (bin count, `lambda_freq`) combination is trained on inner splits of the training states and scored on two inner-validation metrics: configuration-selection agreement with the fold-local expert, and frequency-imitation error in MHz. The selection rule is deliberately conservative: it first keeps only settings whose configuration agreement is within 2 percentage points of the best observed, so tuning the frequency side is never allowed to degrade configuration accuracy, and only among those settings does it pick the smallest bin count whose imitation error is within 5 MHz of the best observed — preferring the simplest adequate model over the most complex one that scores marginally higher. Table 8 shows the per-fold selections; the single agent used for the deployment and latency benchmarks below uses the modal choice across all 10 folds: 64 bins, `lambda_freq` = 0.5.

Held-out state	Bins selected	λ_{freq} selected
x86@10	16	1.00
x86@20	64	2.00
x86@30	16	2.00
x86@40	32	0.25
x86@50	16	0.50
x86@60	32	0.50
x86@70	32	0.25
x86@80	64	0.50
x86@90	64	1.00
x86@100	64	1.00

Table 8. Nested per-fold hyperparameter selections for the Discretised-frequency DDQN. Modal deployment choice: 64 bins, $\lambda_{\text{freq}} = 0.5$.

All six agents use standard offline-RL stabilization techniques: Double-DQN target computation for the three discrete variants (an online network selects the action, a separate target network evaluates it); Polyak-averaged (“soft”) target-network updates after every gradient step, with $\tau = 0.05$; linearly decayed epsilon-greedy exploration from 1.0 to 0.05 over training; and, for the continuous agents, twin critics for TD3 and SAC (to reduce value overestimation the same way Double DQN does for the discrete agents), clipped target-policy-smoothing noise and actor updates delayed to every second critic step for TD3, and an entropy-encouraging exploration term for SAC. Every agent is trained on three independent seeds (7, 42, 123) per fold, so every reported result includes a seed-to-seed variance estimate rather than a single run. No hyperparameter in this stage — network sizes, learning rates, training steps, target τ , epsilon schedule, or tolerance thresholds — was newly tuned for this report; all were carried over unchanged from the original offline-RL modeling notebook, consistent with keeping model behavior unchanged during this merge.

4.6 Results: Regret vs. Oracle, Expert, and Ondemand

Table 9 extends Table 1 with all six RL agents, aggregated per seed and then averaged across the three seeds, with the seed-to-seed standard deviation of mean regret reported alongside.

Policy	Mean regret	Median	Worst	Exact-oracle	Within 5%	Beats ondemand	Seed σ
Oracle	0.00%	0.00%	0.00%	100%	100%	100%	0.000
Configuration Expert / Student	6.93%	3.90%	19.52%	30%	60%	100%	0.000
Discretised-frequency DDQN	14.36%	4.87%	87.33%	33.3%	60.0%	100%	0.014
DDQN	14.43%	4.36%	87.33%	36.7%	56.7%	100%	0.015
Dueling DDQN	14.76%	5.83%	65.82%	33.3%	53.3%	93.3%	0.007
TD3	34.65%	25.55%	117.73%	10.0%	10.0%	60.0%	0.026

Policy	Mean regret	Median	Worst	Exact-oracle	Within 5%	Beats ondemand	Seed σ
DDPG	34.84%	23.30%	135.45%	3.3%	3.3%	60.0%	0.032
SAC	36.36%	25.90%	119.11%	6.7%	6.7%	56.7%	0.044
Linux Ondemand	56.64%	23.37%	365.24%	0%	0%	—	0.000

Table 9. Master comparison, measured EDP, x86, leave-one-state-out, averaged over 3 seeds.

None of the six agents beat the Final Expert (the deployed Configuration Student) outright — a win was defined as beating it by more than 0.5 percentage points of absolute regret, and every agent fell well short. The three discrete agents cluster tightly at 14.4–14.8% mean regret, 7.4–7.8 percentage points behind the expert, and recover between 85% and 92% of the expert's 19.09-percentage-point advantage over ondemand. The three continuous agents cluster at 34.6–36.4% mean regret, 27.7–29.4 percentage points behind the expert, recovering only 6–21% of the expert's advantage. All six agents nonetheless still clear ondemand's 56.64% mean regret by a wide margin, and the discrete agents in particular beat ondemand in 93–100% of held-out states.

Agent	Mean regret	Gap vs. expert
Discretised-frequency DDQN	14.36%	+7.43 pp
DDQN	14.43%	+7.51 pp
Dueling DDQN	14.76%	+7.83 pp
TD3	34.65%	+27.73 pp
DDPG	34.84%	+27.91 pp
SAC	36.36%	+29.43 pp

Table 10. Each RL agent's mean regret compared against the Final Expert (6.93% mean regret); win threshold ± 0.5 percentage points.

4.7 Results: Frequency-Imitation Quality

For the four agents that emit a frequency decision, Table 11 reports how closely that decision tracks both the raw measured frequency and the supervised Frequency Expert's own prediction, which — as Section 4.4 explains — is what these agents are actually trained to imitate. Plain DDQN and Dueling DDQN are absent from this table by construction: they never emit a frequency decision at all.

Model	MAE vs. measured	MAE vs. frequency expert	Legal violations
Discretised-frequency DDQN (64 bins)	131.60 MHz	103.06 MHz	0%
DDPG	44.26 MHz	43.25 MHz	0%
TD3	46.19 MHz	46.51 MHz	0%
SAC	49.76 MHz	45.49 MHz	0%

Table 11. RL frequency-imitation quality. This is an imitation-accuracy metric, never an EDP claim.

The three continuous agents track the frequency expert to within about 43–47 MHz, comparable in kind to the supervised frequency student's own 16.85 MHz figure though noticeably looser, which is expected given

they are optimizing a blended reward rather than frequency prediction alone. The Discretised-frequency DDQN's larger 103.06 MHz gap is a direct, structural consequence of binning: at 64 bins across each architecture's legal frequency range, each bin spans roughly 20 MHz, and the agent's error is dominated by one specific held-out state (x86@100, full load) where all three seeds land on a configuration whose bin centers sit 260–490 MHz away from the frequency expert's prediction for that state, pulling the aggregate figure up substantially. Every frequency decision from every agent, in every held-out state and seed, fell inside the architecture's legal frequency range with zero violations.

4.8 Key Finding: Action-Space Collapse in the Continuous Agents

Inspecting the agents' actual per-state decisions — not just their aggregate regret — reveals a specific, mechanistic explanation for the accuracy gap in Section 4.6. The three discrete agents track the oracle's qualitative direction closely: sampling rate is chosen low (10,000 microseconds) at low-to-mid utilization and scaled up as utilization rises, matching the oracle's own pattern, and their choices are unanimous across all three training seeds in 6 of the 10 held-out states, with disagreement concentrated at the two extreme states ($p \approx 0.10$ and $p \approx 0.98$) — precisely the states responsible for the discrete agents' 65–87% worst-case regret figures in Table 9.

The three continuous agents show a materially different pattern: across all 10 held-out states, DDPG, TD3, and SAC choose a sampling rate of 90,000 or 100,000 microseconds in every single state, never once selecting the low-sampling-rate configurations the oracle prefers at low utilization. In effect, the continuous actor's sampling-rate and threshold outputs stopped responding meaningfully to the utilization input and converged to a near-constant high-sampling-rate policy, while its separately-parameterized frequency output continued to track the frequency expert reasonably well (Table 11). This is the direct, observable cause of the continuous agents' much larger regret: they are not selecting worse configurations at random, they are consistently selecting from a narrow region of the action space that is a poor match for low-and-mid utilization workloads specifically. This is a genuine, actionable finding for follow-up work — see the recommendations in Section 7 — rather than an artifact of the evaluation protocol, since it was confirmed directly from the trained agents' committed per-state actions across all three training seeds.

4.9 Results: Latency

Model	Median latency	p95 latency	p99 latency	Size
DDQN	0.0115 ms	0.0180 ms	0.0296 ms	39.29 KB
Dueling DDQN	0.0181 ms	0.0320 ms	0.0441 ms	21.51 KB
Discretised-frequency DDQN (64 bins)	0.0278 ms	0.0423 ms	0.0542 ms	83.31 KB
DDPG	0.0202 ms	0.0311 ms	0.0468 ms	1.69 KB
TD3	0.0198 ms	0.0279 ms	0.0441 ms	1.69 KB
SAC	0.0202 ms	0.0319 ms	0.0455 ms	1.69 KB

Table 12. RL agent latency, single full action (configuration, plus frequency where applicable).

Every RL agent decides in well under 0.03 milliseconds, faster than either supervised teacher and in the same range as the supervised students. The continuous agents are also the smallest models in the entire report by a wide margin (1.69 KB, just the actor network — the critics used only for training are discarded at deployment), while the Discretised-frequency DDQN is the largest RL agent at 83.31 KB owing to its two-head architecture and 64-bin output.

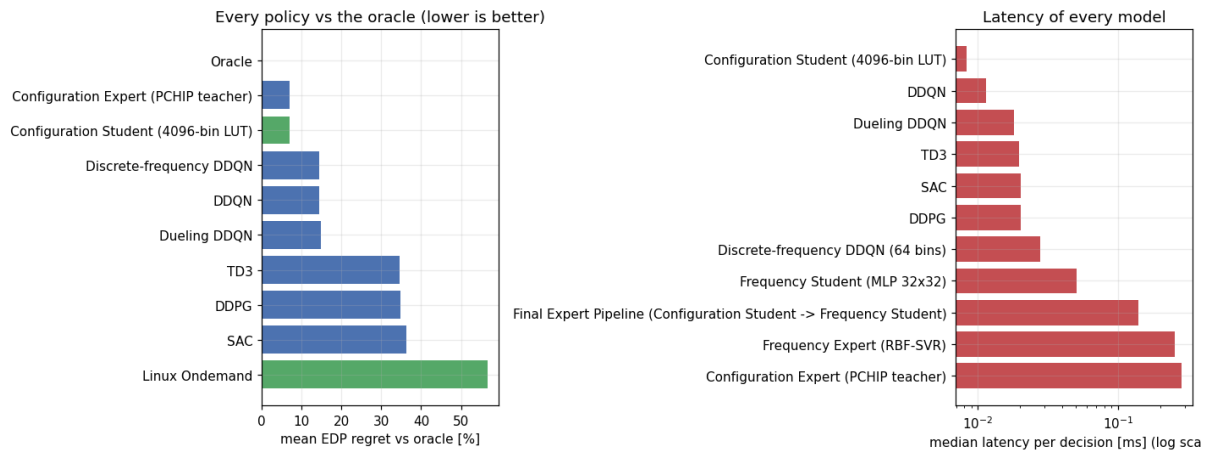


Figure 3. Left: mean EDP regret for every policy (lower is better); the expert/student and ondemand are highlighted. Right: median decision latency for every model in the system, log scale.

5. Consolidated Latency Summary

Table 13 lists every model built in this project, ranked fastest to slowest. Every model, from the fastest RL agent to the slowest supervised teacher, decides in well under half a millisecond, so latency is not a limiting factor for deploying any of the policies discussed in this report; the distilled students and RL agents are simply the more efficient choice where the difference matters.

Model	Median latency	p95 latency	p99 latency	Size
Configuration Student (4096-bin LUT)	0.0083 ms	0.0133 ms	0.0284 ms	36.39 KB
DDQN	0.0115 ms	0.0180 ms	0.0296 ms	39.29 KB
Dueling DDQN	0.0181 ms	0.0320 ms	0.0441 ms	21.51 KB
TD3	0.0198 ms	0.0279 ms	0.0441 ms	1.69 KB
SAC	0.0202 ms	0.0319 ms	0.0455 ms	1.69 KB
DDPG	0.0202 ms	0.0311 ms	0.0468 ms	1.69 KB
Discretised-frequency DDQN (64 bins)	0.0278 ms	0.0423 ms	0.0542 ms	83.31 KB
Frequency Student (MLP 32×32)	0.0503 ms	0.0798 ms	0.0933 ms	13.02 KB
Final Expert Pipeline (both students chained)	0.1401 ms	0.2056 ms	0.2635 ms	49.24 KB
Frequency Expert (RBF-SVR)	0.2549 ms	0.3667 ms	0.4787 ms	28.50 KB
Configuration Expert (PCHIP teacher)	0.2844 ms	0.3856 ms	0.4761 ms	24.93 KB

Table 13. Every model in the system, sorted by median single-decision latency.

6. Validation and Integrity Checks

Nine automated integrity checks run at the end of every execution of the pipeline and are enforced as hard assertions — the run halts rather than silently reporting a broken number if any check fails. All nine passed on the run this report is based on, and the same result was independently reproduced across four separate full end-to-end executions.

#	Check	Result
1	Action grid has exactly 50 (sampling rate, threshold) configurations	Passed
2	Oracle scores exactly zero regret in every fold, by construction	Passed
3	Every predicted frequency (expert and student) obeys the architecture's legal range	Passed
4	The LUT student agrees with its fold-local teacher on $\geq 99.9\%$ of a dense utilization sweep	Passed
5	Attaching the frequency stage never changes the configuration decision (pipeline additivity)	Passed
6	Every RL agent's chosen frequency, where it emits one, is legal	Passed
7	Offline RL uses $\gamma = 0$ (contextual bandit, not sequential control)	Passed
8	Every RL policy was evaluated on all 10 held-out x86 states $\times$ 3 seeds	Passed
9	No held-out x86 state's utilization appears in its own fold's configuration-training data	Passed

Beyond these automated checks, results were cross-validated by independently re-deriving key aggregate figures directly from raw per-state EDP values and by re-running the full pipeline end to end on four separate occasions, which reproduced every accuracy figure in this report bit-for-bit identically each time — all randomness in the system is routed through explicitly seeded generators.

7. Conclusions and Recommendations

The distilled supervised pipeline — the Configuration Student feeding the Frequency Student — is the strongest policy evaluated in this report on every axis: it recovers about 79% of the maximum possible EDP improvement over ondemand, beats ondemand in every held-out state, and completes a full decision (configuration and frequency) in 0.14 milliseconds. It is the recommended default for deployment. Among the RL agents, the three Double-DQN variants are a credible secondary option: none beat the supervised expert, but all three came within roughly 7–8 percentage points of it, beat ondemand in 93–100% of states, and decide even faster than the supervised pipeline (under 0.03 milliseconds). Of the three, the Discretised-frequency DDQN is the most complete standalone alternative, since it is the only RL agent that reproduces the full two-part decision (configuration and frequency) without relying on a separate frequency model.

The three continuous agents (DDPG, TD3, SAC) are not recommended for deployment in their current form. Section 4.8's finding — that all three converged to a narrow, near-constant high-sampling-rate configuration policy rather than tracking utilization — points to a specific, addressable direction for follow-up work: revisiting the actor's exploration schedule and the balance between the configuration-reward gradient and the frequency-imitation gradient during training, since the frequency output continued to track its target well while the configuration output collapsed. Two further directions follow directly from this report's scope notes below: extending EDP (power) measurement to the ARM platform would allow the configuration model, the Final Expert Pipeline, and the offline RL agents to be trained and evaluated on ARM as well as x86; and, since the offline RL agents were trained under $\gamma = 0$ as a contextual bandit, a natural next step if future data captures genuine state-to-state sequencing would be to revisit the same architectures under a nonzero discount factor.